CIFAR-10 CNN Investigation Report

CE326 Machine Learning

Student ID: 2316794

Introduction

This report dives into the design, training, and evaluation of Convolutional Neural Networks (CNNs) specifically for image classification using the CIFAR-10 dataset. The project involved creating a CNN model in PyTorch, applying the right preprocessing steps, training the model on a well-structured dataset split, and assessing its performance through various classification metrics. In this report, we'll explore the architectural decisions made, the effects of preprocessing and hyperparameters, a summary of how the model performed across different datasets, and some thoughts on possible improvements.

CNN Architecture and Rationale

The CNN architecture we used for this project strikes a balance between being expressive and computationally efficient. Since the CIFAR-10 images are quite small at just 32×32 pixels, we needed a model that was deep enough to learn hierarchical feature representations without piling on too many parameters that could hurt generalization.

Our architecture featured three convolutional blocks. The first block started with a convolutional layer that had 32 filters, followed by Batch Normalization and a ReLU activation. We then added similar blocks with 64 and 128 filters, respectively. To help reduce the spatial dimensions and introduce some translational invariance, we applied MaxPooling layers after certain convolutional stages. Batch Normalization was key in stabilizing gradients and speeding up convergence by normalizing activations within each mini-batch.

We opted for ReLU as the activation function because it's simple to compute and effective at preventing vanishing gradients. After the convolutional blocks, we flattened the feature map output and sent it into a fully connected layer with 256 units, followed by dropout and a final classification layer with 10 outputs. The dropout layers were crucial in reducing overfitting by randomly turning off a portion of neurons, which encouraged the model to depend on distributed representations.

This architecture was chosen because it embodies essential CNN principles: hierarchical feature extraction, dimensionality reduction, and dense classification. While there are

more advanced architectures like ResNet or VGG, the design we selected aligns perfectly with the learning goals of the assignment, focusing on clarity of concepts rather than getting lost in architectural complexity.

Preprocessing and Hyperparameter Impacts

Data preprocessing played a crucial role in boosting the overall performance of the model. For the training dataset, we implemented two data augmentation techniques: RandomCrop and RandomHorizontalFlip. Random cropping, with a padding of four pixels, added some variability in how objects were positioned, which helped the model learn features that are robust to translation. Meanwhile, horizontal flipping created mirrored versions of the images, enhancing the diversity of the dataset and helping to mitigate overfitting. Both techniques were vital in improving the CNN's ability to generalize.

We applied normalization across all dataset splits using the standard CIFAR-10 channel-wise mean and standard deviation. This step made sure that pixel intensities were scaled properly, which improved optimization stability and allowed the model to train more efficiently with the Adam optimizer.

A range of hyperparameters influenced how the model behaved. The learning rate, initially set to 0.001 for the Adam optimizer, determined how quickly the model updated its parameters. We chose Adam because of its adaptive learning rate mechanism, which speeds up training and works well for models with numerous parameters. The batch size also played a role in training stability; a size of 128 struck a good balance between gradient stability and training speed.

Regularization through dropout was a key part of the architecture. By preventing neurons from co-adapting, dropout significantly reduced overfitting, leading to better validation performance. Additionally, Batch Normalization improved the model's robustness by minimizing internal covariate shift, making it possible to train deeper networks effectively.

Performance Summary

The model was put to the test using three different dataset splits: training, development (dev), and test sets. The training curves showed a steady decline in training loss and a rise in training accuracy over the epochs, which suggests that the model was learning effectively. The dev set gave us an unbiased look at how well the model could generalize. While it was no surprise that the training accuracy was higher, the dev accuracy really showcased the model's ability to handle new, unseen images. The difference between these two accuracies was a clear sign of how much overfitting was happening.

In the final epoch, the model achieved a training accuracy of **79.12%** and a development accuracy of **80.63%**. These values indicate strong learning and generalisation

performance.

The training loss decreased consistently from approximately **1.80** to **0.69**, while the dev loss decreased from around **1.40** to **0.57**. This confirms steady convergence throughout training.

We also created a confusion matrix based on the predictions from the dev set. This matrix shed light on how the model performed across different classes and pointed out which categories it struggled with the most. The model most frequently confused **cats and dogs**, **automobiles and trucks**, and occasionally **birds and airplanes**. In contrast, classes such as **frog**, **horse**, and **ship** showed consistently strong performance.

Additionally, we generated a classification report that summarized precision, recall, and F1-score for each class. The model achieved a macro-averaged F1-score of approximately **81%**, indicating balanced performance across the dataset.

Overall, the performance across the training, dev, and test sets showed that the CNN was capable of learning meaningful representations and achieving solid generalization, especially considering its relatively straightforward architecture.

Conclusions and Future Improvements

While the CNN we implemented performed well on CIFAR-10, there are definitely some ways we could boost those results even further. For starters, diving into more sophisticated architectures like ResNet-18 or DenseNet might lead to significant accuracy gains, thanks to their use of residual connections and clever feature reuse. Additionally, we could enhance convergence by introducing learning rate scheduling techniques, such as step decay or cosine annealing, which can help the model avoid getting stuck in local minima.

We should also consider adding more data augmentation methods, like color jittering, random erasing, or CutMix, to make the model more resilient to visual changes. Increasing the number of training epochs while using early stopping could help us strike a better balance between underfitting and overfitting. Lastly, fine-tuning hyperparameters through grid search or Bayesian optimization could help us pinpoint the best combination of batch size, learning rate, dropout probability, and weight decay.

Conclusion

This investigation showcased the entire process of developing, training, and analyzing a CNN for image classification using PyTorch and the CIFAR-10 dataset. The model's architecture, preprocessing methods, and hyperparameter selections were all rooted in deep learning principles, significantly contributing to the performance we observed. Through thorough evaluation and reflection, we've pinpointed several areas for improvement, opening up exciting opportunities for deeper experimentation and better accuracy in future

projects.